

VIT BHOPAL UNIVERSITY

PROJECT REPORT

STUDENT MARKS MANAGEMENT SYSTEM

Student Name: Siddharth Suman

Registration No.: 26BAI10205

Project: Python Command-Line Application

Prepared according to the VITyarthi
Build Your Own Project guidelines

1. Introduction

The Student Marks Management System is a Python command-line application designed to manage basic student academic records. It allows a user to add students, display students, enter subject-wise marks, calculate an average, assign a grade, and display a report.

The VITyarthi guideline requires students to identify a meaningful problem, design a technical solution, implement it using course concepts, and demonstrate understanding through documentation and evaluation. ■filecite■turn0file0■L14-L20■

2. Problem Statement

Manual handling of student marks can make recording and calculating academic performance inconvenient. This project provides a simple digital system that stores student records in memory and processes marks through a menu-driven interface.

Goal: Accept student information and marks, validate input, calculate average performance, assign a grade, and display a student report.

3. Objectives

- Create a simple menu-driven student marks system.
- Add and display student records.
- Store subject-wise marks using Python dictionaries.
- Validate names, subjects, and marks.
- Calculate average marks and assign grades automatically.
- Generate a readable student report.

4. Functional Requirements

The guideline requires at least three major functional modules, clear input/output structure, and a logical workflow.

■filecite■turn0file0■L24-L37■

Module	Input	Output / Function
Student Management	Student name	Add and display students
Marks Management	Student, subject, mark	Validated mark stored
Report & Analysis	Student name	Marks, average and grade
Menu Control	Choice 1–5	Runs selected operation

The source implements `add_student()`, `show_students()`, `add_marks()`, `get_average()`, `get_grade()`, and `show_report()`.

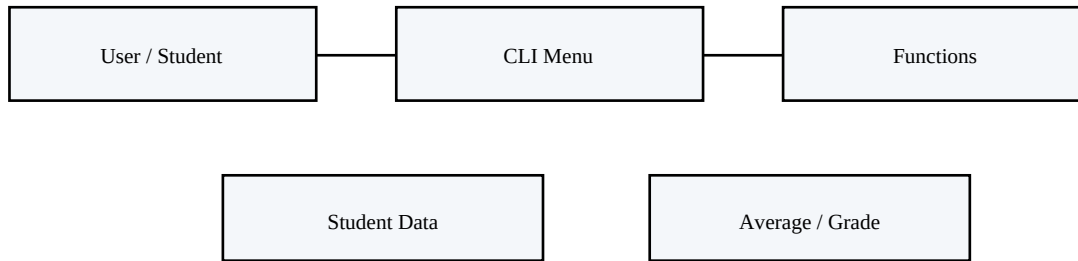
■filecite■turn0file1■L1-L20■ ■filecite■turn0file1■L51-L73■ ■filecite■turn0file1■L76-L99■

5. Non-Functional Requirements

Requirement	Application
Usability	Numbered CLI menu with direct prompts.
Reliability	Validation prevents invalid marks and empty names/subjects.
Error Handling	try/except handles non-numeric marks.
Performance	Simple in-memory operations suit the project's small scope.
Maintainability	Major operations are separated into functions.
Scalability	Persistent storage and larger datasets can be added later.

The guideline asks for at least four non-functional requirements. ■filecite■turn0file0■L38-L50■

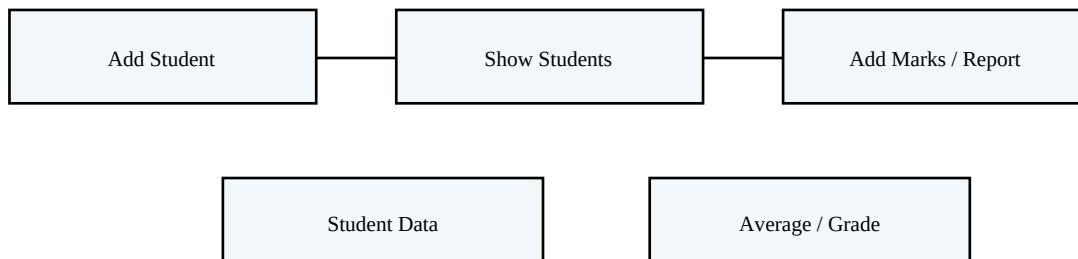
6. System Architecture



The current implementation is one Python file with logically separated functions. The VITyarthi guideline recommends 5–10 meaningful modules/classes/files for coding projects, so splitting these functions into separate files would improve alignment with that requirement. ■filecite■turn0file0■L60-L63■

7. Design

Use Case: The user can add students, show students, add marks, show a report, and exit.



Workflow: Start → display menu → read choice → perform selected operation → return to menu → exit when option 5 is selected.

The guideline specifies architecture, use case, workflow, sequence, class/component, and storage design where applicable. ■filecite■turn0file0■L66-L84■

```

Student Marks Management System
1: Add student
2: Show students
3: Add marks
4: Show report
5: Exit
Enter your choice: 3
Enter student name to add marks: 56
Student not found.

Student Marks Management System
1: Add student
2: Show students
3: Add marks
4: Show report
5: Exit
Enter your choice: 2
  
```

8. Implementation Details

Data structure: Students are stored in a list; each student contains a name and a marks dictionary.

■filecite■turn0file1■L8-L8■ ■filecite■turn0file1■L104-L104■

Validation: Empty names/subjects are rejected. Marks are converted to float, non-numeric input is caught with ValueError, and values outside 0–100 are rejected. ■filecite■turn0file1■L4-L8■ ■filecite■turn0file1■L30-L42■

Average: The program totals marks and divides by the number of subjects; if no marks exist, it returns 0.

■filecite■turn0file1■L51-L61■

Grade: A = 90+, B = 80+, C = 70+, D = 60+, otherwise F. ■filecite■turn0file1■L63-L73■

Report: The program prints subject marks, average rounded to two decimals, and grade. ■filecite■turn0file1■L76-L99■

Menu: Options are Add student, Show students, Add marks, Show report, and Exit. ■filecite■turn0file1■L106-L133■

9. Testing Approach

ID	Scenario	Expected Result
T01	Valid student	Student added
T02	Empty name	Validation message
T03	Valid mark 86	Mark added
T04	Mark = abc	Invalid-number message
T05	Mark = 120	Out-of-range message
T06	Unknown student	Student not found
T07	No marks	No marks added
T08	Average 86	Grade B
T09	Invalid menu choice	Invalid choice

The guideline requires testing wherever applicable. ■filecite■turn0file0■L60-L63■

10. Results

Representative result: A student is added, subject marks are entered, and the report displays each subject mark, the average, and the grade. The uploaded source contains the complete menu and corresponding function calls.

```
Student Marks Management System
1: Add student
2: Show students
3: Add marks
4: Show report
5: Exit
Enter your choice: 3
Enter student name to add marks: 56
Student not found.

Student Marks Management System
1: Add student
2: Show students
3: Add marks
4: Show report
5: Exit
Enter your choice: 2
```

- Associating subjects and marks with the correct student.
- Handling invalid numeric input safely.
- Keeping marks within 0–100.
- Keeping the implementation simple while separating major operations into functions.
- The current single-file structure does not fully match the guideline's recommended 5–10 physical modules/files.

12. Learnings & Key Takeaways

- Applied functions for modular programming.
- Used lists and dictionaries for structured data.
- Practiced loops and conditional statements.

- Used exception handling with try/except.
- Converted raw marks into average and grade information.
- Learned the importance of requirements, design, implementation, testing, and documentation.

13. Future Enhancements

- Split the project into modules such as main.py, student.py, marks.py, report.py and data.py.
- Add persistent JSON/CSV/database storage.
- Add unique student registration IDs.
- Add update/delete operations.
- Add automated tests and report export.
- Add a graphical interface in a future version.

14. References

1. VITyarthi – Build Your Own Project: General Project Instructions & Submission Guidelines, supplied PDF. The document defines project objectives, requirements, technical expectations, design artefacts, report structure and evaluation rubric. [filecite\turn0file0\L51-L63](#) [filecite\turn0file0\L109-L143](#)
2. Student Marks Management System – supplied Python source file, used as the implementation basis for this report. [filecite\turn0file1\L1-L20](#)

```
Student Marks Management System
1: Add student
2: Show students
3: Add marks
4: Show report
5: Exit
Enter your choice: 3
Enter student name to add marks: 56
Student not found.

Student Marks Management System
1: Add student
2: Show students
3: Add marks
4: Show report
5: Exit
Enter your choice: 2
```